

DEPUTY: A DPU-Based Network Offloading Architecture with Minimal CPU Involvement for Stable Network Performance

Yuchan Lee¹, Soho Jang¹, Sookwang Lee², Shinyoung Ahn², Jaehwan Lee¹

1 Korea Aerospace University

2 Electronics and Telecommunications Research Institute



Korea
Aerospace
University



Distributed Infrastructure
for Data Intelligence Lab.

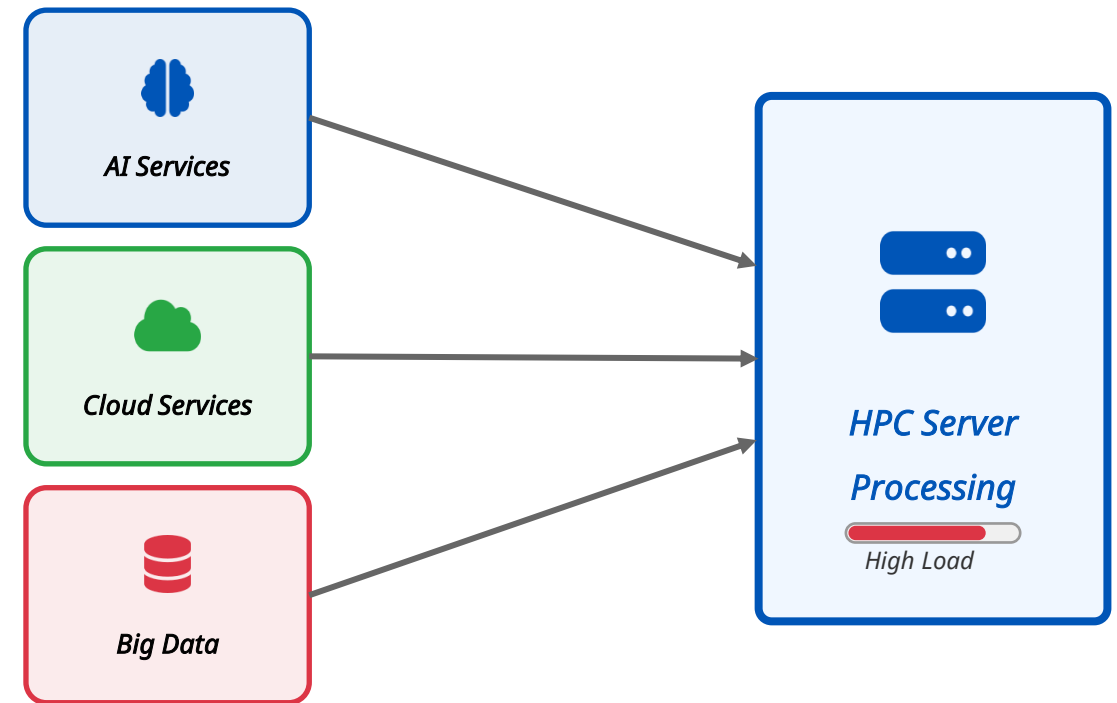


Electronics and Telecommunications
Research Institute



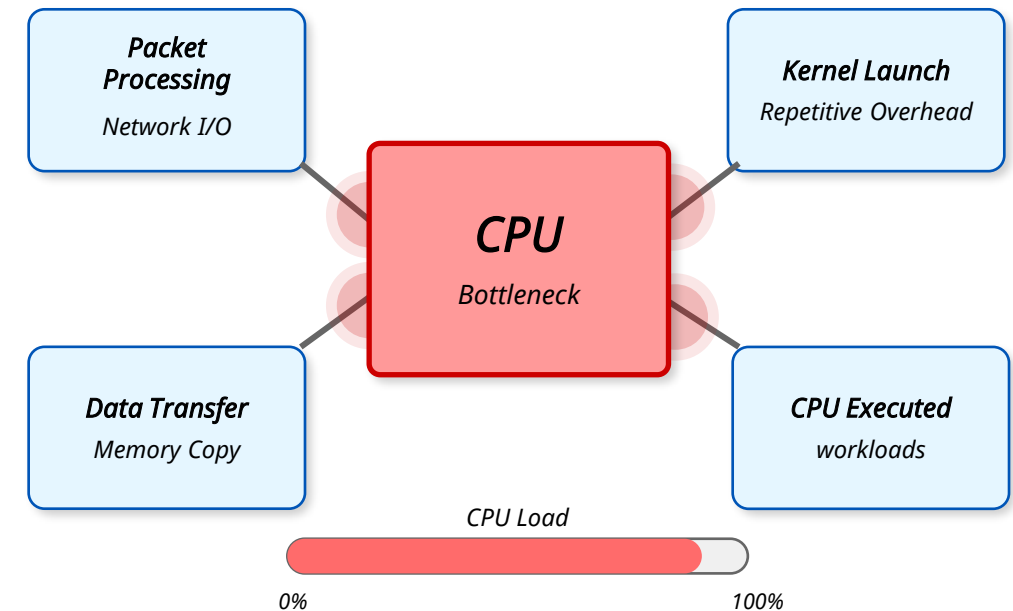
Research Background

- **Increasing demand** for real-time Big Data processing due to growth of Cloud/HPC and large-scale AI services
- While GPUs handle computation, **frequent CPU intervention** occurs in network processing and data movement
- CPU-centric structures cause increased latency, resource inefficiency, and higher power consumption
- Advancement of high-performance NICs and GPUs -> **Increased control flow/data** volume that CPU must manage



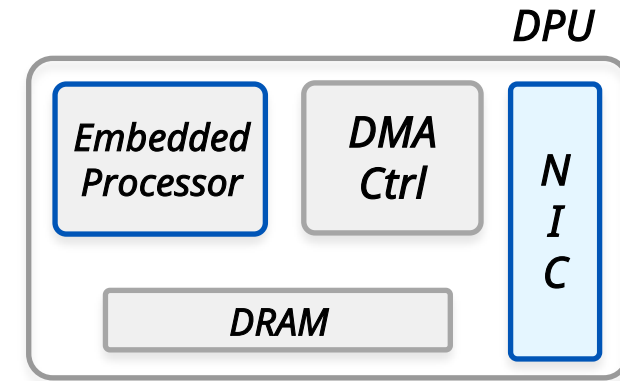
Limitations of CPU-Centric Architecture

- Packet processing, data transfer, and kernel launches all concentrated on the CPU -> **bottleneck**
- **Accumulated overhead** due to repetitive CPU-GPU communication when executing multiple short kernels
- Increased latency variability and reduced service stability in high-load environments
- **Requirement: Minimize CPU involvement, end-to-end offloading of network-inference pipeline**

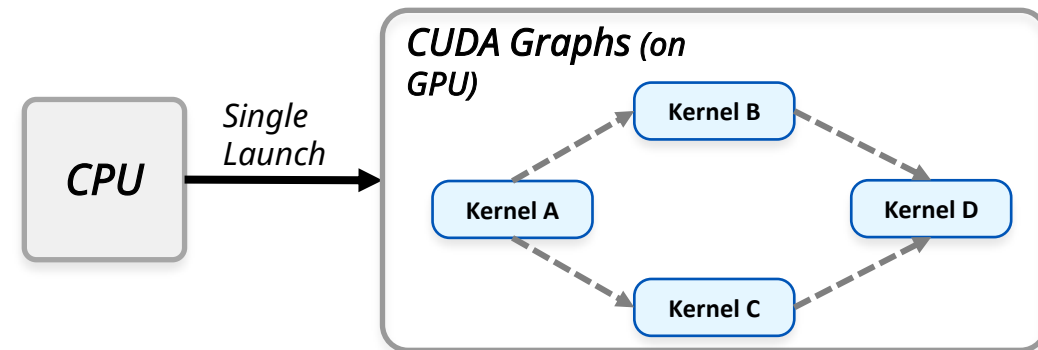


Technical Background

- **DPU(Data Processing Unit):** NIC + embedded processor, offloading infrastructure tasks from the CPU

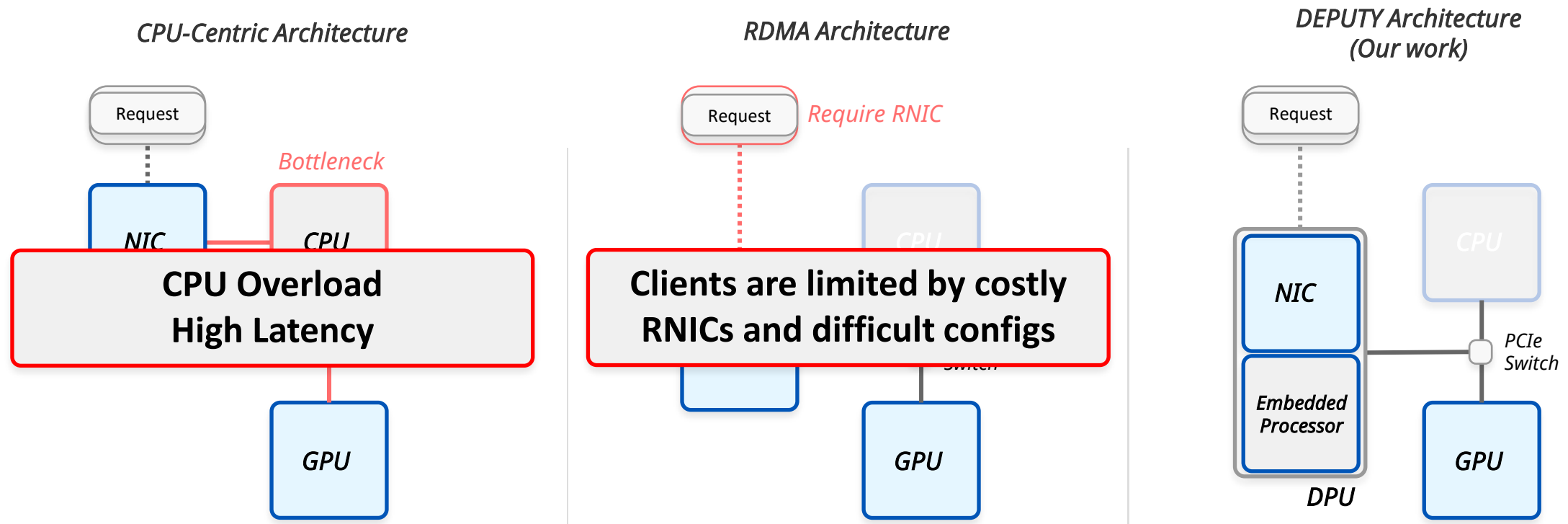


- **CUDA Graphs:** Bundles GPU operations into one graph for a single launch



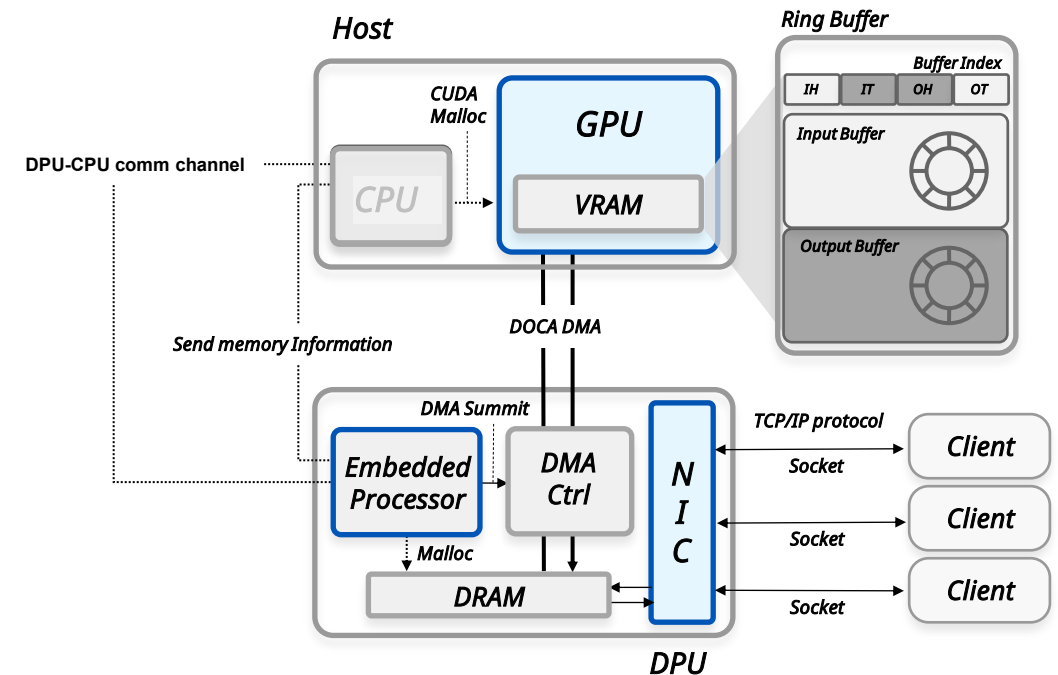
Comparison of Existing Solutions

- CPU-Centric: NIC->CPU->GPU copy path, **CPU overload**, high latency
- RDMA(GPUDirect): Direct NIC->GPU transfer possible but **requires RNIC on both server/client**, deployment constraints
- DEPUTY: **DPU handles all networking**, DPU->GPU direct transfer using DOCA DMA, clients use **standard sockets**



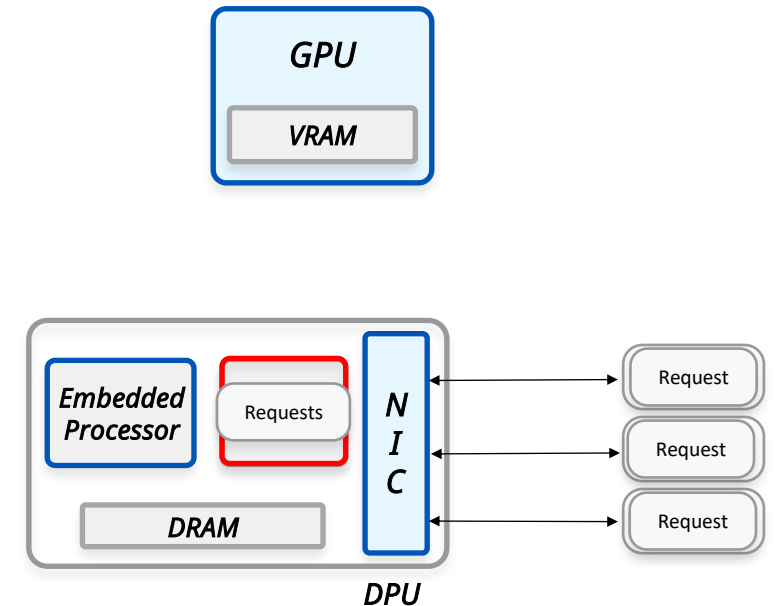
System Architecture

- > DPU: Socket handling, TCP/IP protocol, payload reception/transmission, DOCA DMA submission
- > Path: Client <-> DPU <- DOCA DMA -> GPU(VRAM)
- > Shared structure: Input/Output Rings in GPU VRAM with head/tail indices (IH, IT, OH, OT)
- > CPU: Only involved in initialization phase, **excluded from data path during execution**



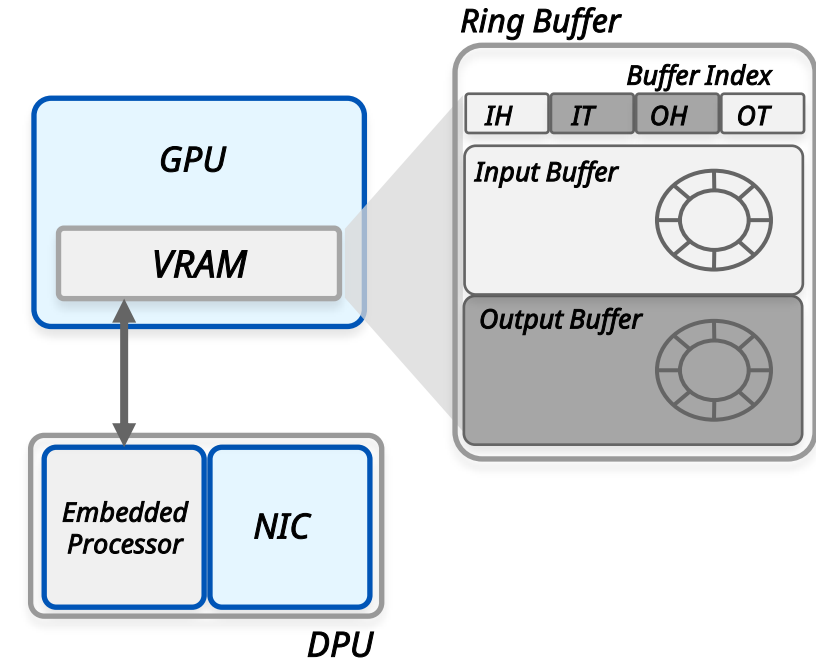
Core Technology 1: Network Offloading & DOCA DMA

- **DPU(BlueField-3) handles** all socket connections, network protocols, and payload management
- DOCA DMA: Hardware-accelerated DMA library
- Asynchronous Execution: Parallel processing of network operations and DMA tasks improves system efficiency
- Advantages over RDMA: Eliminates doorbell/MR management overhead, no need for client RNIC



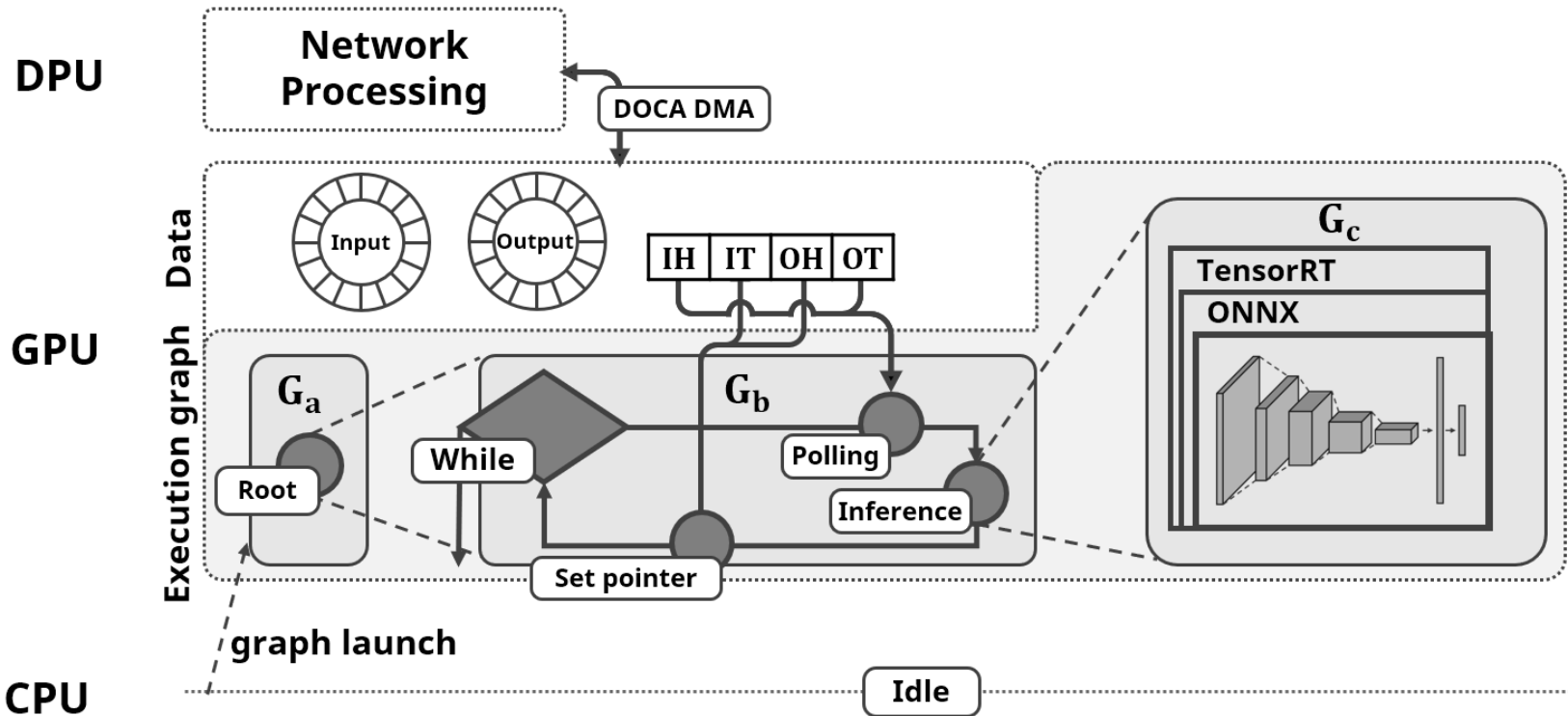
Core Technology 2: Ring Buffer-based Data Structure

- Location: In GPU VRAM - **Minimizing data access** path in inference-latency dominated environments
- Structure: Head/tail index-based polling - Calculating new entries **without scanning the entire buffer**
- Access Control: Separating DPU write and GPU write regions to **prevent conflicts**
- Batch Processing: Improving throughput by **batching data** using head-tail difference



Core Technology 3: CUDA Graphs-based Autonomous Execution

- > Pipeline: Converting from PyTorch -> ONNX -> TensorRT -> CUDA Graphs
- > Higher-level Graph (G_a): Loop control with while node, **CPU launches only once**
- > Graph Structure: Integrating polling, pointer updates, and inference kernels into a single loop body (G_b)
- > Benefits: Elimination of kernel launch overhead, continuous GPU execution, **no CPU intervention**



Experimental Setup



Server

2×Intel Xeon E5-2690 v4 @ 2.60GHz (28 cores/56 threads)
Memory: 251 GB



Accelerators

4×NVIDIA A100 GPU (40 GB VRAM)
NVIDIA BlueField-3 DPU (16 Arm-cores, 32GB DDR5)



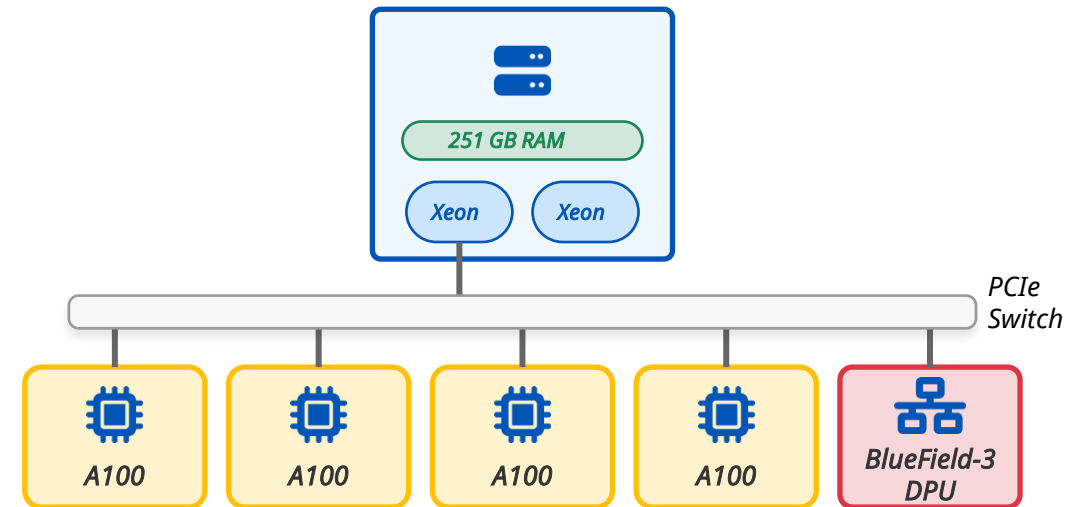
Interconnect

InfiniBand, IPoIB for client communication



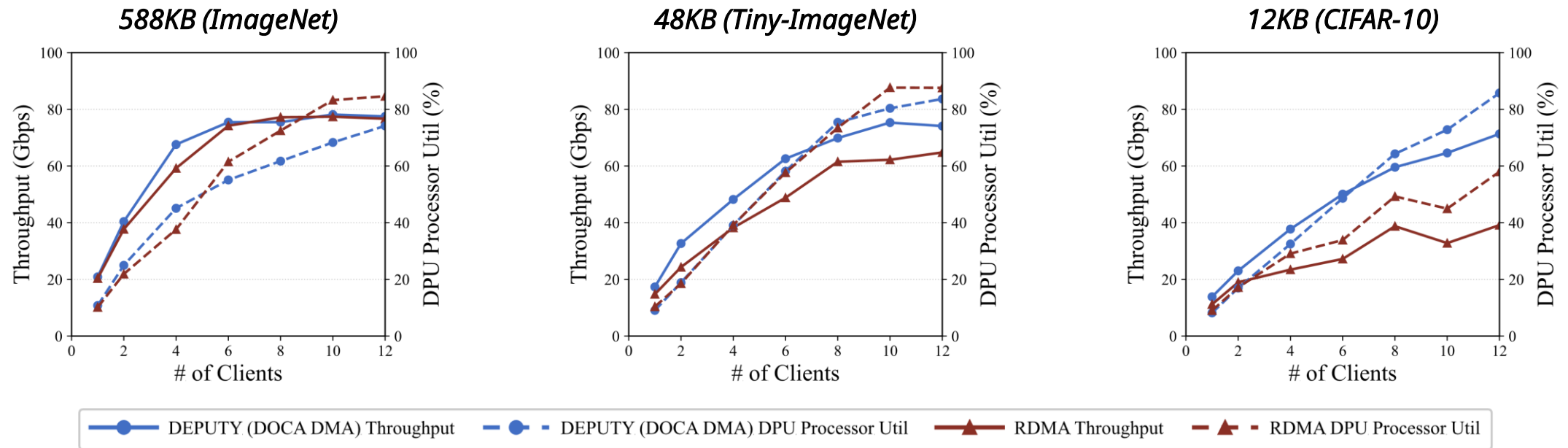
Software & Measurement Methods

Ubuntu 22.04, DOCA 2.8.0, CUDA 12.4
10s warm-up, 50 iterations, median value used
Total system CPU utilization measured



Performance Evaluation 1: DPU→GPU Transfer (DOCA DMA vs RDMA)

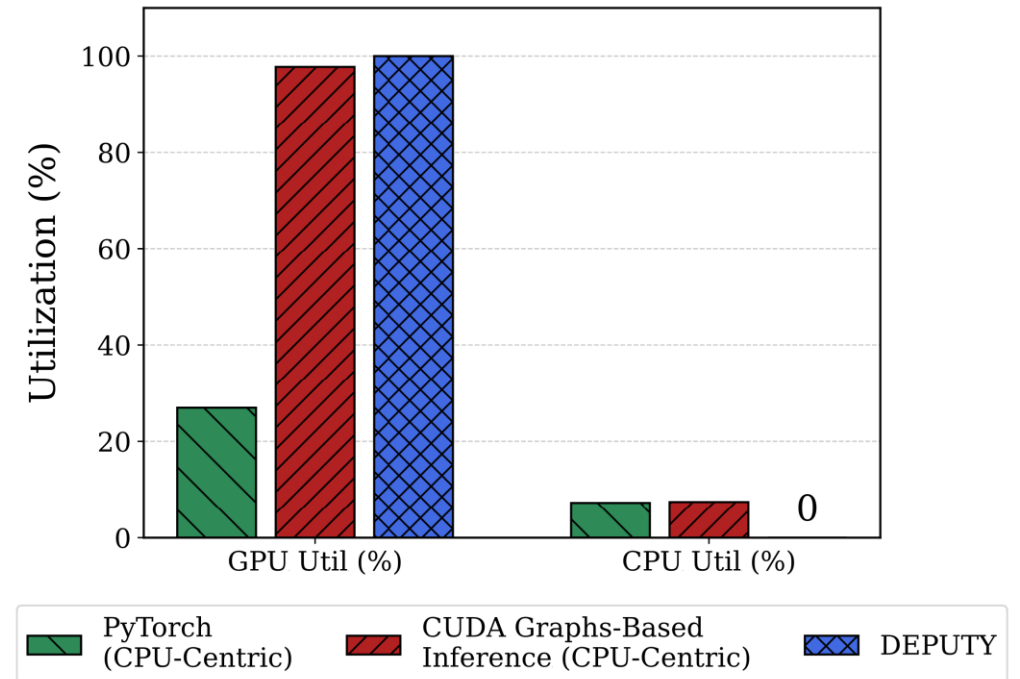
- Buffer sizes: 588KB (ImageNet), 48KB (Tiny-ImageNet), 12KB (CIFAR-10)
- Large buffer (588KB): Both methods achieve ~80 Gbps (embedded processor limit)
- Small buffers (48/12KB): using DOCA DMA achieves up to **1.96× higher throughput** than using RDMA
- Analysis: RDMA protocol overhead (doorbell/MR management) becomes dominant with small buffers



Performance Evaluation 2: Overall System Performance

- Throughput (Images/s): DEPUTY 1702.37 | CUDA Graphs 1697.32 | PyTorch 139.84
- DEPUTY achieves 12.17× higher throughput than PyTorch by eliminating CPU intervention

System	Throughput (Images/s)
DEPUTY	1702.37
CUDA Graph-Based (CPU-Centric)	1697.32
PyTorch (CPU-Centric)	139.84



Conclusion and Contributions



Contributions:

- (1) **DPU-based full network offloading and data path design**
- (2) **Inference Execution without CPU Intervention**
- (3) **Quantitative performance evaluation**



Achievements:

Up to 1.96× higher throughput compared to existing approach
CPU utilization ~0%, GPU utilization 100%



Practicality:

Standard socket-based client compatibility



Future Work:

Support for diverse workloads and deployment environments

Thanks for your attention! Questions?

Contact

- Yuchan Lee / lyuch0424@kau.kr
- Jaehwan Lee / jlee@kau.ac.kr
- Distributed & Big Data Computing Lab / <http://bigdata.kau.ac.kr>



Korea
Aerospace
University

